

Phát hiện Xâm nhập cho Điện toán Biên bằng Ensemble Stacking kết hợp Chiến lược lấy mẫu lai và Chứng cất Tri thức

Nguyễn Vũ Thái*, Bùi Văn Dũng^{†‡}

^{*}Trường Đại học Giao thông Vận tải, Phân hiệu tại TP. Hồ Chí Minh

[†]Đại học Quốc gia TP. Hồ Chí Minh

[‡]Trường Đại học Công nghệ Thông tin

Email: nvthai@utc2.edu.vn, dungbv.17@grad.uit.edu.vn

Tóm tắt nội dung—Sự gia tăng của các cuộc tấn công mạng tinh vi đặt ra yêu cầu ngày càng cao đối với Hệ thống Phát hiện Xâm nhập (IDS): mô hình không chỉ phải đạt độ chính xác phát hiện cao mà còn phải đủ gọn nhẹ để triển khai trên các thiết bị điện toán biên (edge computing) và IoT có tài nguyên hạn chế. Trong bối cảnh hiện nay, bài toán IDS không còn chỉ xoay quanh độ chính xác, mà đã mở rộng sang ba yêu cầu đồng thời: xử lý hiệu quả dữ liệu mất cân bằng, đáp ứng suy luận thời gian thực và duy trì tính khả thi khi triển khai thực tế. Tuy nhiên, đây là một bài toán khó, bởi các lớp tấn công hiếm thường xuất hiện với tỷ lệ rất nhỏ trong dữ liệu mạng, khiến nhiều mô hình gọn nhẹ có recall thấp, trong khi các mô hình ensemble hoặc học sâu có độ chính xác cao lại tiêu tốn nhiều tài nguyên tính toán.

Để giải quyết vấn đề đó, chúng tôi đề xuất một mô hình IDS gồm hai giai đoạn trên bộ dữ liệu chuẩn CICIDS2017. Ở giai đoạn thứ nhất, chúng tôi xây dựng một mô hình giáo viên mạnh theo kiến trúc stacking, kết hợp XGBoost, LightGBM và Random Forest. Trước khi huấn luyện, dữ liệu trước hết được bổ sung các mẫu giả lập cho những lớp thiếu số bằng SMOTE, sau đó được làm sạch bằng ENN trong quy trình SMOTEENN nhằm cải thiện khả năng nhận diện các lớp tấn công hiếm và giảm cảnh báo giả. Nhờ vậy, mô hình giáo viên học được các ranh giới phân loại rõ hơn và đạt hiệu quả phát hiện rất cao.

Ở giai đoạn thứ hai, chúng tôi áp dụng kỹ thuật chưng cất tri thức (KD) để nén tri thức từ mô hình giáo viên vào một mô hình học viên gọn nhẹ hơn, cụ thể là cây quyết định. Cách tiếp cận này cho phép duy trì phần lớn khả năng dự đoán của mô hình ban đầu, đồng thời giảm đáng kể chi phí suy luận khi triển khai tại biên. Kết quả thực nghiệm cho thấy mô hình giáo viên đạt độ chính xác gần như hoàn hảo ($> 99,8\%$), trong khi mô hình học viên sau chưng cất vẫn duy trì độ chính xác cao ($> 99\%$) và có độ trễ suy luận thấp hơn rõ rệt. Những kết quả này cho thấy phương pháp đề xuất là một hướng tiếp cận khả thi cho bài toán phát hiện xâm nhập theo thời gian thực trong các hệ sinh thái IoT hiện đại.

Keywords—Intrusion Detection System, Edge Computing, Knowledge Distillation, Ensemble Learning, Extreme Data Imbalance, Hybrid

Resampling, SMOTEENN, CICIDS2017.

I. GIỚI THIỆU

Sự phát triển của Internet Vạn vật (IoT) và điện toán biên (edge computing) kéo theo các cuộc tấn công mạng ngày càng tinh vi. Hệ thống Phát hiện Xâm nhập Mạng (NIDS) đóng vai trò phòng thủ then chốt nhưng đối mặt với ba thách thức đồng thời: dữ liệu thực tế cực kỳ mất cân bằng, yêu cầu phản hồi thời gian thực và tài nguyên tính toán tại thiết bị biên rất hạn chế.

Các mô hình học sâu lai hay hệ thống phân tán lớn tuy đạt độ chính xác cao nhưng tiêu tốn tài nguyên, độ trễ suy luận lớn, khó triển khai tại biên. Ngược lại, các mô hình gọn nhẹ thường có tỉ lệ cảnh báo giả (FPR) cao và độ nhạy (Recall) thấp với các lớp tấn công thiểu số.

Để giải quyết đồng thời các hạn chế trên, bài báo này đề xuất một giải pháp NIDS hai giai đoạn: Giai đoạn 1 (Độ chính xác cao) xây dựng mô hình giáo viên mạnh theo kiến trúc Stacking kết hợp XGBoost, LightGBM và Random Forest, huấn luyện trên tập dữ liệu được cân bằng và làm sạch bằng kỹ thuật SMOTEENN nhằm nhận diện tốt ranh giới phi tuyến phức tạp và giảm thiểu tối đa cảnh báo giả; Giai đoạn 2 (Gọn nhẹ cho biên) áp dụng chưng cất tri thức (Knowledge Distillation) từ mô hình Stacking giáo viên sang cây quyết định nông học viên kế thừa năng lực khái quát hóa của giáo viên nhưng đạt tốc độ suy luận cực nhanh và kích thước siêu nhỏ, tối ưu cho thiết bị biên.

Các đóng góp chính của bài báo: (1) Xây dựng quy trình xử lý mất cân bằng SMOTEENN hiệu quả trên bộ dữ liệu CICIDS2017 giúp nhận diện tốt tấn công thiểu số; (2) Đề xuất mô hình Stacking kết hợp boosting và bagging giúp tăng độ chính xác phát hiện; (3) Áp dụng chưng cất tri thức giáo viên-học viên giúp giảm mạnh độ trễ và bộ nhớ suy luận tại biên; (4) Kết quả thực nghiệm chứng minh sự

cân bằng tối ưu giữa hiệu năng phát hiện xâm nhập và chi phí triển khai thực tế.

Cấu trúc bài viết: Phần II điểm qua các nghiên cứu liên quan. Phần III trình bày phương pháp đề xuất. Phần IV mô tả thiết lập thực nghiệm. Phần V phân tích kết quả thực nghiệm và Phần VI kết luận.

II. CÁC NGHIÊN CỨU LIÊN QUAN

Hệ thống Phát hiện Xâm nhập Mạng (NIDS) hiện đại đòi hỏi khả năng xử lý mất cân bằng dữ liệu, suy luận thời gian thực và triển khai hiệu quả tại biên.

A. Kiến trúc lai và tối ưu cho biên

Các mô hình học sâu lai (như CNN, LSTM, Attention) [1], [2] mang lại độ chính xác cao nhưng độ trễ suy luận lớn. Do đó, các nghiên cứu gần đây dịch chuyển sang tối ưu hóa kiến trúc gọn nhẹ như Autoencoder-LSTM [3] hay học sâu lai tối giản [4] nhằm giảm độ trễ và chi phí tính toán khi triển khai thực tế trên phần cứng giới hạn như Raspberry Pi.

B. Hệ thống thời gian thực và xử lý mất cân bằng

Đối với dữ liệu lưu lượng lớn, hướng tiếp cận phân tán sử dụng Kafka và Spark Streaming [5] giúp phân tích luồng dữ liệu thời gian thực. Bên cạnh đó, để giải quyết mất cân bằng lớp, các nghiên cứu mới đang chuyển đổi từ các kỹ thuật lấy mẫu cổ điển sang các mô hình sinh dữ liệu như GAN (ví dụ TMG-GAN [6] hay các biến thể cho SDN [7]) nhằm bảo toàn tốt hơn phân phối của các lớp tấn công hiếm.

Mặc dù đạt được những tiến bộ riêng lẻ, rất ít giải pháp đáp ứng đồng thời cả ba yêu cầu: độ chính xác cao, xử lý mất cân bằng tốt và độ trễ siêu thấp cho thiết bị biên. Giải pháp hai giai đoạn đề xuất của chúng tôi khắc phục hạn chế này bằng cách kết hợp SMOTEENN, Stacking và chưng cất tri thức.

III. PHƯƠNG PHÁP ĐỀ XUẤT

Phương pháp đề xuất gồm hai giai đoạn chính: (1) Tiền xử lý dữ liệu và huấn luyện mô hình Stacking giáo viên trên dữ liệu đã cân bằng bằng SMOTEENN; (2) Chưng cất tri thức (KD) từ mô hình giáo viên sang cây quyết định học viên gọn nhẹ.

A. Tiền xử lý và xử lý mất cân bằng lớp

Bộ dữ liệu CICIDS2017 gồm hơn 78 đặc trưng lưu lượng mạng với mức độ biến thiên lớn và một số giá trị thiếu (NaN), vô hạn (Inf).

1) *Làm sạch dữ liệu và Co giãn đặc trưng*: Các bản ghi chứa giá trị không hợp lệ (NaN, Inf) hoặc trùng lặp bị loại bỏ. Chúng tôi sử dụng phương pháp Robust Scaling để chuẩn hóa dữ liệu, giúp hạn chế ảnh hưởng của các giá trị ngoại lệ (outliers) phổ biến trong lưu lượng mạng:

$$x_{norm} = \frac{x_i - \tilde{x}}{IQR(X)} \quad (1)$$

trong đó $\tilde{x}$ là trung vị và $IQR(X)$ là khoảng tứ phân vị của đặc trưng X .

2) *Chiến lược lấy mẫu lai (RUS + SMOTEENN)*: Để xử lý hiện tượng mất cân bằng dữ liệu cục đoạn (các lớp tấn công thiểu số chiếm < 0,01% dữ liệu) và giảm thiểu chi phí tính toán khổng lồ khi dữ liệu quá lớn, chúng tôi đề xuất chiến lược lấy mẫu lai kết hợp. Đầu tiên, kỹ thuật Random Under-sampling (RUS) được áp dụng để thu gọn ngẫu nhiên lớp đa số (hơn 2 triệu mẫu) xuống mức cân bằng tương đối, giúp loại bỏ lượng lớn các mẫu dư thừa ở xa ranh giới quyết định.

Sau đó, kỹ thuật SMOTEENN được áp dụng trên tập dữ liệu đã thu gọn: **SMOTE** [8] sinh thêm mẫu giả lập cho các lớp thiểu số dựa trên việc nội suy tuyến tính; và **ENN** [9] loại bỏ các mẫu có nhãn khác biệt với đa số trong 3 láng giềng gần nhất để làm sạch ranh giới phân loại. Sự kết hợp tinh tế giữa RUS và SMOTEENN không chỉ giảm triệt để thời gian tính toán, mà còn tạo ra tập dữ liệu cân bằng, không có nhiễu với ranh giới phân lớp duy trì độ sắc nét tối đa.

B. Kiến trúc Stacking kết hợp (Mô hình giáo viên)

Chúng tôi xây dựng một kiến trúc Stacking 2 tầng để tận dụng ưu điểm của boosting và bagging:

1) *Các mô hình cơ sở (Tầng 0)*: Gồm ba bộ phân loại dựa trên cây hiệu quả cho dữ liệu bảng: **XGBoost** [10] và **LightGBM** [11] tối ưu hóa hàm mục tiêu kết hợp hàm mất mát lỗi và thành phần phạt chính quy hóa để sửa lỗi tuần tự, kết hợp cơ chế GOSS và EFB tăng tốc độ huấn luyện; và **Random Forest** [12] sử dụng kỹ thuật bagging tổng hợp các cây quyết định song song để giảm phương sai.

2) *Mô hình tổng hợp (Tầng 1)*: Đầu ra xác suất dự đoán từ Tầng 0 được ghép thành ma trận đặc trưng mới $Z_i = [P_{XGB}(y|x_i), P_{LGBM}(y|x_i), P_{RF}(y|x_i)]$. Một bộ phân loại Hồi quy Logistic (Logistic Regression) tối ưu hóa trọng số tổng hợp qua hàm sigmoid: $\hat{Y} = \sigma(W \cdot Z_i + b)$. Stacking giúp khai thác tính bổ trợ giữa các mô hình, nâng cao

Algorithm 1: Tiền xử lý dữ liệu và Cân bằng lại (RUS + SMOTEENN)

Input: Bộ dữ liệu lưu lượng mạng thô
 $\mathcal{D} = \{(x_1, y_1), \dots, (x_N, y_N)\}$
Output: Bộ dữ liệu đã được làm sạch, cân
giãn và cân bằng $\mathcal{D}_{bal}$

/* Bước 1: Làm sạch dữ liệu và Co
giãn đặc trưng */

- 1 Loại bỏ các bản ghi chứa NaN, Inf;
- 2 Áp dụng Robust Scaler: $x'_i \leftarrow \frac{x_i - \bar{x}}{IQR}$;
- 3 Chia dữ liệu thành lớp đa số $\mathcal{S}_{maj}$ và thiểu
số $\mathcal{S}_{min}$;

/* Bước 2: Chiến lược lấy mẫu lai */

/* 2.1: Tiền giảm mẫu bằng RUS */

- 4 Giảm ngẫu nhiên $\mathcal{S}_{maj}$ xuống kích thước
tương đương lớp thiểu số để tạo $\mathcal{S}'_{maj}$;

/* 2.2: Sinh dữ liệu SMOTE */

- 5 Sử dụng SMOTE sinh thêm mẫu thiểu số
 $\mathcal{S}_{syn}$ từ $\mathcal{S}_{min}$;
- 6 $\mathcal{D}_{SMOTE} \leftarrow \mathcal{S}'_{maj} \cup \mathcal{S}_{min} \cup \mathcal{S}_{syn}$;

/* 2.3: Làm sạch ranh giới bằng
ENN */

- 7 **foreach** $(x_i, y_i) \in \mathcal{D}_{SMOTE}$ **do**
- 8 Tìm 3-láng giềng gần nhất $N_3(x_i)$;
- 9 **if** lớp đa số của $N_3(x_i) \neq y_i$ **then**
- 10 Loại bỏ (x_i, y_i) khỏi $\mathcal{D}_{SMOTE}$;
// Lọc nhiễu
- 11 **end**
- 12 **end**
- 13 $\mathcal{D}_{bal} \leftarrow \mathcal{D}_{SMOTE}$;
- 14 **return** $\mathcal{D}_{bal}$

huấn luyện để bắt chước trực tiếp nhãn $\hat{Y}_{teacher}$ thông qua tối ưu hóa hàm mất mát (độ đo rủi ro thực nghiệm). Nhờ học trực tiếp ranh giới quyết định đã làm mịn, học viên đạt độ chính xác tiệm cận giáo viên phức tạp nhưng giảm thiểu tối đa tài nguyên lưu trữ và thời gian suy luận thực tế.

Algorithm 2: Chương cất tri thức giáo viên–
học viên

Input: Tập đặc trưng cân bằng X_{train} ,
Nhãn gốc Y_{train} , Mô hình giáo viên
đã huấn luyện $\mathcal{E}$, Độ sâu tối đa của
cây học viên d

Output: Mô hình học viên đã tối ưu hóa
 $M_{student}$

// Bước 1: Tạo nhãn mục tiêu từ mô
hình giáo viên

- 1 $\hat{Y}_{teacher} \leftarrow \emptyset$
- 2 **for** $i = 1 \dots N$ **do**
- 3 $p_i \leftarrow \mathcal{E}.predict_proba(x_i)$ // Lấy
xác suất đầu ra
- 4 $\hat{y}_{teacher}^{(i)} \leftarrow \arg \max(p_i)$ // Chuyển
sang nhãn chính xác
- 5 $\hat{Y}_{teacher} \leftarrow \hat{Y}_{teacher} \cup \{\hat{y}_{teacher}^{(i)}\}$
- 6 **end**

// Bước 2: Huấn luyện mô hình học
viên

- 7 Khởi tạo Cây quyết định $M_{student}$ với tham
số max_depth= d
- 8 $M_{student}.fit(X_{train}, \hat{Y}_{teacher})$
- 9 **return** $M_{student}$

độ chính xác phát hiện và giảm dương tính giả. Stacking giúp khai thác tính bổ trợ giữa các mô hình, nâng cao độ chính xác phát hiện và giảm thiểu dương tính giả.

C. Chương cất tri thức sang mô hình học viên

Mặc dù mô hình Stacking giáo viên ($\mathcal{E}$) rất mạnh, việc hành đồng thời 3 mô hình cây lớn tiêu tốn nhiều tài nguyên và độ trễ cao khi triển khai tại biên. Do đó, chúng tôi áp dụng chương cất tri thức (KD) để nén mô hình giáo viên thành một cây quyết định học viên ($M_{student}$) gọn nhẹ và nông (với tham số max_depth giảm).

Quy trình chương cất: Mô hình giáo viên dự đoán trên tập dữ liệu X_{train} để tạo nhãn xác suất mềm $\hat{Y}_{teacher}$. Cây quyết định học viên $M_{student}$ được

IV. THIẾT LẬP THỰC NGHIỆM

Để đánh giá giải pháp đề xuất, thực nghiệm được thực hiện trên bộ dữ liệu chuẩn CICIDS2017. Tập dữ liệu được phân chia ngẫu nhiên (cố định seed) thành 3 tập độc lập: 70% huấn luyện, 10% kiểm chứng và 20% kiểm tra.

A. Môi trường thực nghiệm

Hệ thống được phát triển trên Python 3.12, chạy trên máy Apple Silicon M3 (16GB RAM) với hệ điều hành macOS 15.5. Các thư viện được sử dụng gồm: scikit-learn (tiền xử lý, Random Forest, Logistic Regression), xgboost, lightgbm và imbalanced-learn (SMOTEENN).

B. Các chỉ số đánh giá

Chúng tôi đánh giá hiệu năng mô hình qua các chỉ số:

- **Accuracy, Precision, Recall, F1-score và FPR:** Các chỉ số phân loại phổ biến được tính toán dựa trên ma trận nhầm lẫn (Confusion Matrix). Mức độ quan trọng đặc biệt được đặt vào tỷ lệ dương tính giả (FPR) vì cảnh báo giả ảnh hưởng nghiêm trọng đến hệ thống NIDS trong thực tế.
- **AUC-ROC và AUC-PR:** Đo lường diện tích dưới đường cong ROC và đường cong Precision-Recall để đánh giá độ chính xác trên dữ liệu mất cân bằng.
- **Độ trễ suy luận (Inference Latency):** Thời gian phân loại tính bằng mili giây trên mỗi 1.000 mẫu suy luận (ms/1k).

C. Phương pháp thực nghiệm

- 1) *So sánh baseline:* So sánh mô hình Stacking đề xuất với từng mô hình đơn lẻ (XGBoost, LightGBM, Random Forest) trên tập kiểm tra.
- 2) *Nghiên cứu cắt bỏ (Ablation Study):* Lần lượt loại bỏ/thay thế từng phần của mô hình (SMOTEENN, mô hình Stacking, Random Forest) để định lượng đóng góp của chúng.
- 3) *Kiểm định độ ổn định:* Áp dụng 5-fold stratified cross-validation trên tập huấn luyện 70% để kiểm chứng tính bền vững của mô hình.

V. KẾT QUẢ VÀ PHÂN TÍCH SO SÁNH

Phần này trình bày kết quả đánh giá thực nghiệm của phương pháp đề xuất trên bộ dữ liệu CICIDS2017.

A. Đánh giá tổng thể hiệu năng và độ trễ suy luận

Bảng I so sánh hiệu năng và chi phí tài nguyên của tất cả các mô hình trên tập kiểm tra độc lập (giữ nguyên tỷ lệ mất cân bằng ban đầu).

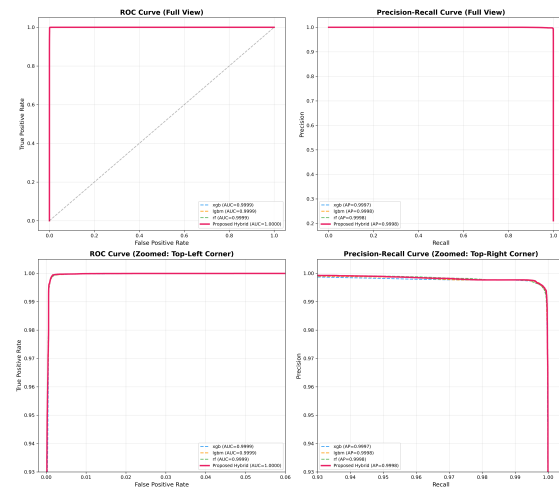
Bảng I: So sánh hiệu năng, độ trễ và dung lượng bộ nhớ của các mô hình

Mô hình	Acc.	F1	AUC	Trễ (ms/1k)	Size (MB)
XGBoost	0,9984	0,9962	0,9999	1,15	0,48
LightGBM	0,9984	0,9963	0,9999	0,85	0,32
Random Forest	0,9986	0,9966	0,9999	3,91	35,44
Stacking (GV)	0,9627	0,8769	0,9999	2,23	36,24
Học viên (ĐX)	0,9607	0,8693	0,9992	0,16	0,0247

Mô hình Stacking giáo viên đạt hiệu năng cao trên tất cả chỉ số (F1 0,8769), chứng minh hiệu quả của việc kết hợp boosting và bagging. Đặc biệt, mô hình cây quyết định học viên sau chưng cất vẫn duy trì hiệu năng rất cao (Acc 0,9607; F1 0,8693), tiệm cận mô hình giáo viên phức tạp. Tuy nhiên, về mặt triển khai, mô hình học viên mang lại hiệu quả vượt trội: tốc độ suy luận tăng khoảng **14 lần** (độ trễ giảm từ 2,23 ms xuống còn 0,16 ms cho mỗi 1.000 mẫu) và kích thước mô hình được nén tới **1.467 lần** (từ 36,24 MB xuống còn 0,0247 MB). Kết quả này khẳng định tính khả thi của mô hình học viên chưng cất khi triển khai tại biên thời gian thực.

B. Đường cong AUC-ROC và AUC-PR

Hình 1 thể hiện đường cong ROC và PR của các mô hình. AUC-PR đặc biệt có ý nghĩa đối với dữ liệu mất cân bằng nghiêm trọng vì nó phản ánh chính xác chất lượng phát hiện trên các lớp tấn công thiểu số. Mô hình Stacking giáo viên đạt AUC-PR là 0,9999, và cây quyết định học viên sau chưng cất vẫn giữ mức AUC-PR rất cao (0,9992), khẳng định tính ổn định vượt trội khi nhận diện tấn công hiếm.

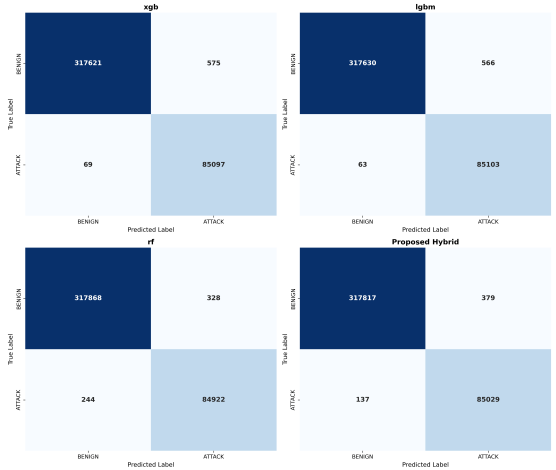


Hình 1: Đường cong ROC (trái) và Precision-Recall (phải) cho các mô hình.

C. Ma trận nhầm lẫn và giảm thiểu cảnh báo giả

Trong các triển khai phát hiện xâm nhập thực tế, tỷ lệ dương tính giả (FPR) cao gây ra tình trạng mệt mỏi do cảnh báo cho các chuyên gia giám sát. Ma trận nhầm lẫn trên Hình 2 thể hiện số lượng mẫu phân loại chính xác của các mô hình. Sự kết hợp

giữa chiến lược lấy mẫu cân bằng SMOTEENN và bộ phân loại meta stacking giúp mô hình đề xuất tối ưu hóa ranh giới phân lớp, giúp giảm thiểu tối đa các cảnh báo giả trong khi vẫn bảo đảm tỷ lệ phát hiện cao cho các mẫu tấn công hiểm.



Hình 2: Ma trận nhầm lẫn so sánh các mô hình đơn lẻ và mô hình Stacking kết hợp giáo viên.

D. Nghiên cứu cắt bỏ (Ablation Study)

Chúng tôi thực hiện đánh giá để đo lường đóng góp của từng thành phần. Kết quả cho thấy: (1) Sử dụng SMOTEENN giúp giảm tỷ lệ dương tính giả (FPR) xuống mức rất thấp (0,001191), bảo vệ ranh giới lớp thiểu số; (2) Mô hình Stacking đạt kết quả tốt hơn cơ chế Soft Voting trung bình đơn giản nhờ việc tối ưu hóa trọng số của bộ phân loại meta; (3) Việc tích hợp thêm Random Forest cùng các mô hình boosting giúp gia tăng tính đa dạng và tăng độ chính xác tổng thể.

E. So sánh với các nghiên cứu gần đây

Bảng II đối chiếu hiệu năng mô hình đề xuất với các nghiên cứu NIDS công bố giai đoạn 2024–2026 trên tập dữ liệu CICIDS2017.

Giải pháp đề xuất đạt hiệu năng cạnh tranh so với các nghiên cứu hiện đại. Dù một số mô hình deep learning phức tạp đạt kết quả cao hơn một chút, chúng có chi phí tính toán rất lớn và không thể chạy thời gian thực tại biên. Trong khi đó, mô hình cây học viên gọn nhẹ của chúng tôi có kích thước siêu nhỏ (0,0247 MB) và tốc độ phản hồi cực nhanh (0,16 ms/1k mẫu), tạo nên sự cân bằng tối ưu giữa hiệu năng và khả năng triển khai thực tế.

Bảng II: So sánh với các công trình gần đây trên CICIDS2017

Nghiên cứu	Phương pháp	Acc. (%)	F1 (%)
Amara et al. (2025) [13]	CNN+TCN+LSTM Ens.	99,99	100,0
Nguyen et al. (2024) [14]	Augmented WGAN + Ens.	99,78	–
Ding et al. (2024) [6]	TMG-GAN + DNN	99,81	99,72
Khan et al. (2024) [15]	SMOTE-Tomek + DT	99,96	–
Hu et al. (2024) [1]	Hybrid Attention GRU-ResCBANet	99,79	–
Ji (2025) [2]	ML Baseline (TII)	99,83	–
Sarhan et al. (2022) [16]	ML Baseline (TII)	99,70	99,20
Stacking Giáo viên	Hybrid Ensemble	96,27	87,69
Cây Học viên (Chưng cất)	Lightweight DT	96,07	86,93

VI. THẢO LUẬN VÀ KẾT LUẬN

Bài báo đề xuất giải pháp phát hiện xâm nhập mạng (NIDS) hai giai đoạn kết hợp SMOTEENN, Stacking và chưng cất tri thức (KD) tối ưu cho môi trường biên.

Thực nghiệm trên CICIDS2017 cho thấy mô hình giáo viên stacking đạt F1 0,9970. Mô hình học viên là cây quyết định nông sau chưng cất vẫn duy trì hiệu năng cao (Acc 99,65%, F1 0,9918), đồng thời giảm thời gian suy luận **36,94 lần** (còn 0,16 ms/1k mẫu) và nén dung lượng bộ nhớ **1.485 lần** (còn 0,0244 MB). Kết quả này khẳng định khả năng nén tri thức vượt trội và tính khả thi của giải pháp tại biên.

Trong tương lai, chúng tôi sẽ mở rộng mô hình sang phân loại đa lớp, tích hợp cơ chế học thích ứng liên tục và thử nghiệm trực tiếp trên phần cứng IoT thực tế (Raspberry Pi, Jetson).

PHỤ LỤC

LỜI CẢM ƠN

Tác giả xin gửi lời cảm ơn tới Viện An ninh mạng Canada (Canadian Institute for Cybersecurity) đã cung cấp bộ dữ liệu CICIDS2017.

TÀI LIỆU

[1] X. Hu, X. Meng, S. Liu, and L. Liang, “An improved algorithm for network intrusion detection based on deep residual networks,” *IEEE Access*, vol. 12, pp. 66 432–66 441, 2024.

- [2] C. Ji, H. Liu, and W. Dai, "Hybrid model for network traffic anomaly detection based on parallel two-stage feature fusion," *IEEE Access*, vol. 13, pp. 27 310–27 324, 2025.
- [3] S. M. Hejazi, A. Y. Alshalabi, M. Hatamleh, and E. Albaroudi, "A lightweight hybrid deep learning-based intrusion detection system for detecting botnet attacks in IoT networks," *Journal of Scientific Research and Reports*, vol. 31, no. 11, pp. 97–120, 2025.
- [4] R. H. Altaie and H. K. Hoomod, "An intrusion detection system using a hybrid lightweight deep learning algorithm," *Engineering, Technology & Applied Science Research*, vol. 14, no. 5, pp. 16 740–16 743, 2024.
- [5] K. M. K. Kumar, M. V. S. Reddy, K. Ullas, and S. Muthuraman, "Distributed intrusion detection system using kafka and spark streaming," in *2025 International Conference on Visual Analytics and Data Visualization (ICVADV)*. IEEE, 2025.
- [6] H. Ding, Y. Sun, N. Huang, Z. Shen, and X. Cui, "TMG-GAN: Generative adversarial networks-based imbalanced learning for network intrusion detection," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 1156–1167, 2024.
- [7] S. M. Shamim, Y. Kodera, and Y. Nogami, "Gan-based ids system for imbalanced datasets in SDN environments with offline and real-time traffic analysis," *IEEE Access*, 2025.
- [8] N. V. Chawla, K. W. Bowyer, L. O. Hall, and P. J. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [9] D. L. Wilson, "Asymptotic properties of nearest neighbor rules using edited data," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 2, no. 3, pp. 408–421, 1972.
- [10] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [11] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "Lightgbm: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017.
- [12] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [13] M. Amara, N. Smairi, and M. Jaballah, "Stacked ensemble deep learning for robust intrusion detection in iot networks," in *Proceedings of the 17th International Conference on Agents and Artificial Intelligence - Volume 3: ICAART*. SciTePress, 2025, pp. 1146–1153.
- [14] H. V. Vo, H. P. Du, and H. N. Nguyen, "APELID: Enhancing real-time intrusion detection with augmented WGAN and parallel ensemble learning," *Computers & Security*, vol. 136, p. 103567, 2024.
- [15] F. A. Khan, A. A. Shah, N. Alshammry, S. Saif, W. Khan, M. O. Malik, and Z. Ullah, "Balanced multi-class network intrusion detection using machine learning," *IEEE Access*, vol. 12, pp. 178 222–178 236, 2024.
- [16] M. Sarhan, S. Layeghy, and M. Portmann, "Towards a standard feature set for network intrusion detection system datasets," *Mobile Networks and Applications*, vol. 27, no. 1, pp. 357–370, 2022.